

WAPH-Web Application Programming and Hacking - Hackathon 3

Instructor: Dr. Phu Phung

Student

Name: Aiden Ward

Email: ward3aj@mail.uc.edu

Short-bio: Aiden Ward has a lot of interest in computer science, specifically data science. I also love volleyball and cars.



Figure 1: Aiden's headshot

Repository Information

Repository's URL: <https://github.com/aidenward6/waph-ward3aj/tree/main/waph-ward3aj/labs/hackathon3>

Overview

For this hackathon, I performed a session hijacking attack on a vulnerable web application using XSS. I had to create a malicious link that stole the victim's session cookie and then use that cookie to hijack the admin account. This was a really cool way to see how serious XSS can be and why we need to worry about sanitizing inputs. I learned how to use Netcat to capture data and how to manually inject cookies into the browser to bypass authentication.

Part I: The Attack

Demonstration Video On Github

Steps Performed: 1. I injected an XSS payload into the blog comment section. 2. I set up a Netcat listener on my VM to catch the incoming request. 3. As the victim, I clicked the link, which triggered the script to send the cookie to my server. 4. I retrieved the encoded cookie string from the Netcat log and decoded it to find the PHPSESSID. 5. I injected the stolen PHPSESSID into my browser's console using `document.cookie` to gain admin access.

Part II: Understanding and Prevention

Vulnerabilities Exploded: The attack worked because the blog didn't clean the user comments at all. It let me put in raw html with an `onclick` event. Because there was no output encoding, the browser just ran my script when the link was clicked. Also, the app didn't use the `HttpOnly` flag on its cookies. If that was turned on, my javascript wouldn't have been able to read the cookie value, and the whole attack would have failed.

Prevention Mechanisms: To fix this, the dev should use output encoding so that inputs are treated as text and not code. They should also definitely set the `HttpOnly` and `Secure` flags on all session cookies to stop javascript from stealing them. Finally, adding a strong Content Security Policy would help block inline scripts, which would stop this kind of XSS dead in its tracks. It's important to always sanitize inputs before putting them back on the page.